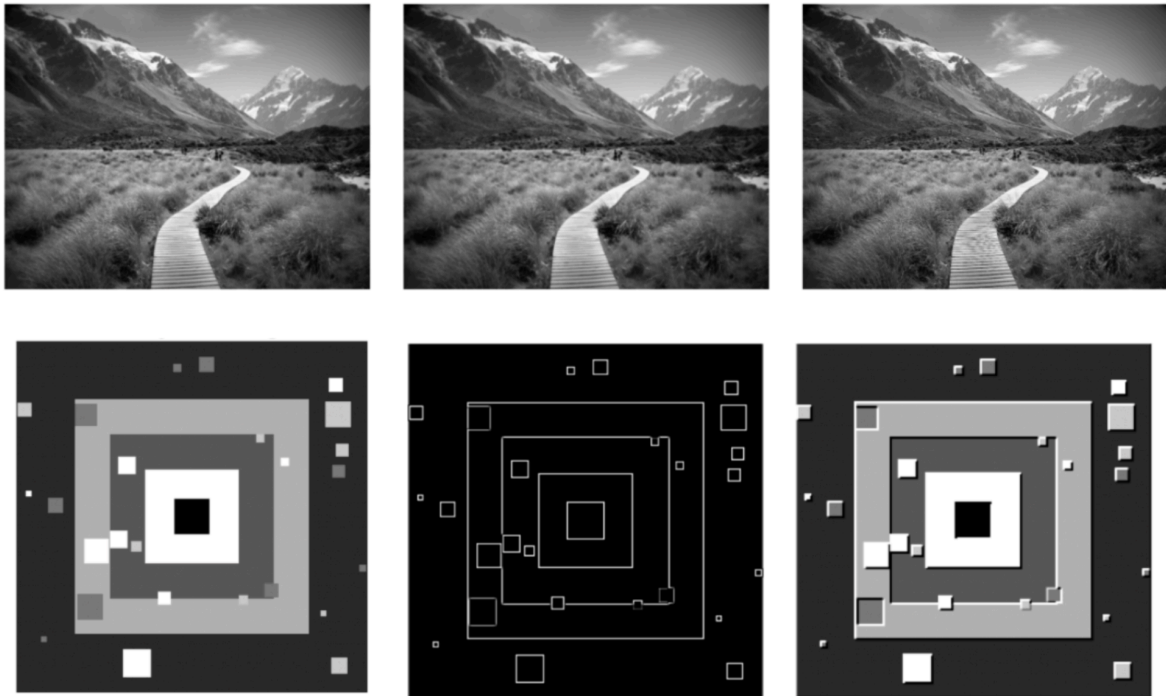


A Comparative Analysis of Image Processing Implementations



Nadav Izhaki - 329087852

Department of Computer Science,

Holon Institute of Technology (HIT).

Course: Classical Image Processing.

05.04.2026

Keywords: Computational Efficiency, Algorithm Optimization, Vectorization, Look-Up Table (LUT), Spatial Convolution

Abstract

Digital image processing requires a constant balance between mathematical precision and computational efficiency. Transitioning from algorithmic theory to practical implementation exposes complex engineering challenges, such as handling boundary conditions, data-type conversions, and scalability under load. This project conducts a comparative analysis of different implementation paradigms for both point and spatial image operations. Algorithms for binarization and gamma correction were evaluated using nested loops, vectorization, and Look-Up Tables (LUT). Additionally, a custom convolution function was implemented, incorporating Zero-Padding and pixel range clipping mechanisms, which was then benchmarked against OpenCV library functions. Performance tests demonstrate that utilizing vectorization and LUTs drastically improved execution times for point operations, reducing the average runtime from approximately 220ms to merely 2-3ms. In the spatial convolution analysis, a dilated binary heatmap revealed localized mathematical discrepancies exclusively at the image boundaries due to differing padding strategies. Furthermore, a stress test utilizing a large 15x15 kernel exhibited severe execution slowdowns in the custom implementation, which operates at $O(K^2)$ complexity, contrasting sharply with the stable performance of hardware-optimized OpenCV functions. The findings illustrate that while custom implementations provide crucial transparency into the "black box" of image processing, large-scale computer vision applications necessitate the integration of highly optimized industrial libraries to maintain practical efficiency.

Introduction

Background and Gap in Existing Knowledge: Digital image processing relies on a diverse set of mathematical operations, ranging from basic pixel-level transformations (such as thresholding and gamma correction) to complex spatial filtering. However, the widespread use of ready-made software libraries often creates a "black box" that obscures the internal mechanics of these algorithms. Consequently, there is a knowledge gap regarding efficient computational resource management, data-type conversions, and the handling of algorithmic edge cases.

Research Justification and Importance: Unboxing these "black boxes" is critical for computer vision developers. A profound understanding of the performance disparities between various programming approaches—such as using basic iterative loops versus array vectorization and Look-Up Tables (LUT)—enables optimal algorithm selection. Furthermore, familiarity with these algorithmic nuances prevents computational bottlenecks and hidden calculation errors in practical applications.

Research Question: How does the choice of implementation paradigm impact the execution efficiency of both point and spatial image operations, and how do custom-built algorithms compare in terms of mathematical accuracy and performance against industry-optimized libraries like OpenCV?

Methods

This section details the implementation process of the image processing algorithms, divided into point-based and spatial operations.

- **Development Environment:** The implementations were written in Python, utilizing the `NumPy` library for array processing, `OpenCV` for baseline performance comparisons, and the `time` module for precise execution benchmarking.
- **Part A - Point Operations Implementation:** Binarization and gamma correction were implemented using three distinct algorithmic approaches to evaluate their computational efficiency:
 - **Nested Loops:** A direct, iterative approach processing each pixel individually. For binarization, a pre-allocated zero-filled array was updated to 255 only when the pixel intensity exceeded the defined threshold. For the gamma operation, strict data type conversion (casting) was applied: original `uint8` values were normalized to a `float32` range of [0.0, 1.0] to

maintain mathematical precision, scaled back by 255.0, and securely cast back to `uint8` to prevent data truncation or overflow.

- **Vectorization:** Leveraging NumPy's built-in capabilities to apply operations across the entire image matrix simultaneously. This included using boolean indexing (`img > threshold`) for binarization, and vectorized power operations on normalized matrices for gamma correction.
- **Look-Up Tables (LUT):** A pre-computed 1D array of 256 elements was generated, containing the final processed value for every possible grayscale intensity. The input image matrix was then used strictly as indices to extract the mapped results instantaneously, entirely bypassing real-time mathematical computations.
- **Part B - Spatial Operations Implementation:** A custom convolution function (`my_imfilter`) was developed using a sliding window approach:
 - **Boundary Conditions:** Dynamic Zero-Padding was applied based on the kernel's dimensions (e.g., a 1-pixel border for a 3x3 kernel) to ensure proper computation at the image edges.
 - **Overflow Control:** To handle negative or out-of-bounds values generated by derivative-based filters (like Laplacian), the sum of products was accumulated using floating-point types. A clipping operation was subsequently applied to strictly bound the values to the legal [0, 255] range before the final `uint8` conversion.
- **Performance Benchmarking:** To ensure a reliable comparative analysis, a systematic benchmark test was established utilizing the `time.perf_counter()` function. Each function variant was executed for 10 consecutive iterations to calculate an average runtime, mitigating environmental system noise, and was compared against the hardware-optimized library functions provided by OpenCV. Furthermore, a "stress test" evaluated complexity scaling using a 15x15 kernel, and a Dilated Binary Heatmap was generated to geometrically visualize accuracy discrepancies at the image boundaries caused by differing padding strategies.

RESULTS

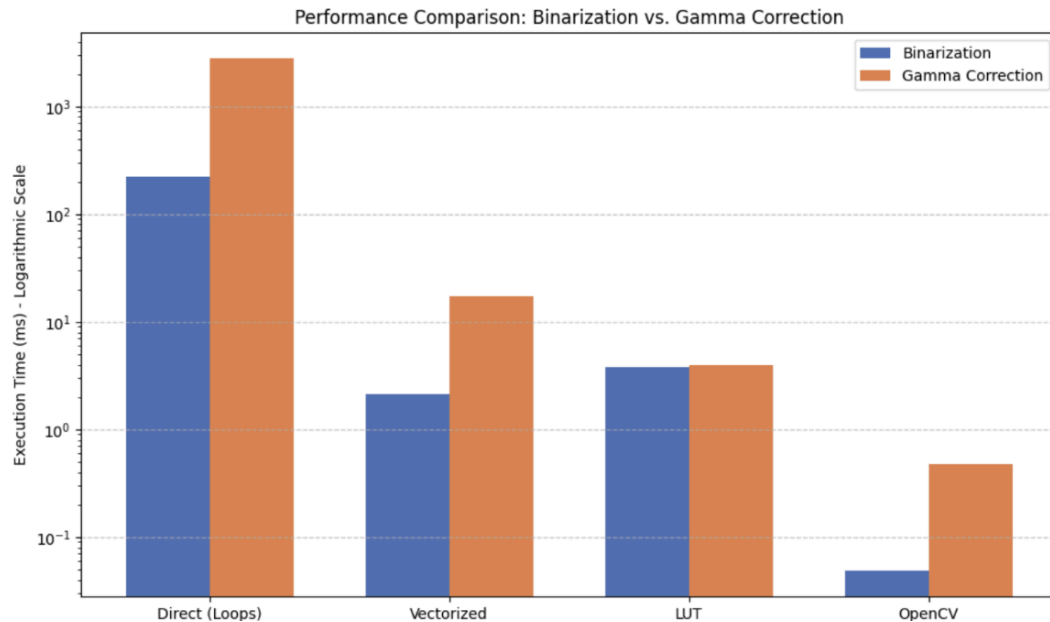
The following quantitative and visual findings evaluate the implementation methods under different conditions using two input types: a real photographic image and a complex synthetic image with sharp gradients.

5.1 Point Operations: Binarization and Gamma Correction

- **Visual Output:** . Mathematically and visually, the pixel output obtained from all three custom implementations (nested loops, vectorization, and LUT) was strictly identical for both Binarization and Gamma Correction

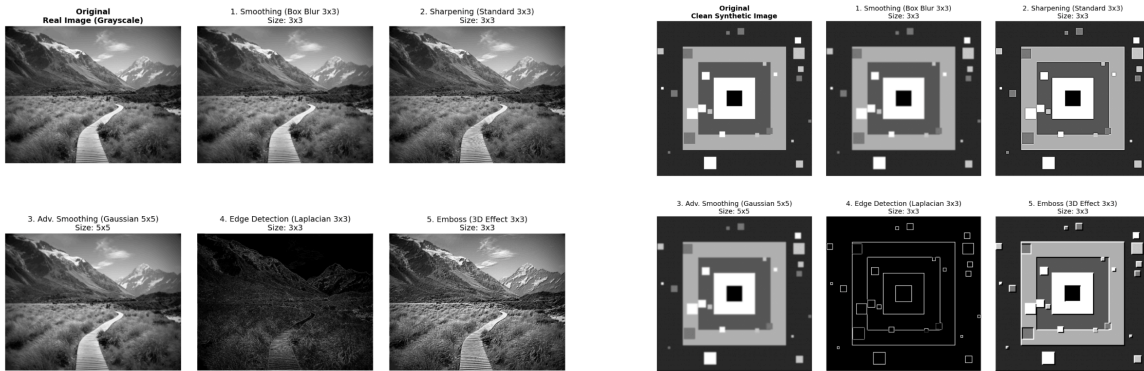


- **Execution Time Benchmarks:**. The combined graph summarizes the average execution times (in milliseconds, ms) over 10 iterations for each implementation method. The data reveals substantial absolute disparities: the nested loops approach took over 200 ms, whereas the vectorization and Look-Up Table (LUT) methods reduced the execution time to the 2-4 ms range. The built-in [OpenCV](#) library function exhibited the shortest runtime (less than 0.1 ms).

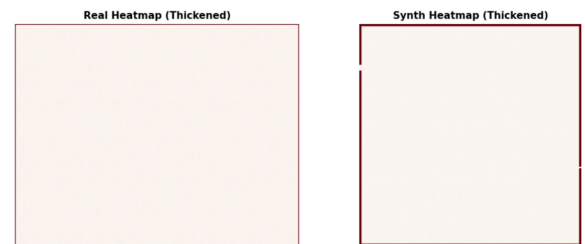


Spatial Operations: Convolution

- **Visual Output:** alongside the 5 spatial filter outputs: Box, Sharpen, Gaussian, Laplacian, Emboss here]. The filters were successfully applied to both input types, demonstrating their distinct effects on fine details and edges.



- **Accuracy and Discrepancies (Sanity Check):** The absolute difference map between the custom `my_imfilter` (using Zero-Padding) and the library function `cv2.filter2D` (using reflection padding) showed that non-zero differences occurred exclusively at the image boundaries. The core of the image yielded a complete mathematical match (zero discrepancies). The thickness of this discrepancy border matched the kernel's radius exactly.



- **Stress Test and Computational Complexity:** The stress test demonstrated that scaling the kernel size from 3x3 to 15x15 resulted in a significant exponential increase in the execution time of the custom `my_imfilter`. Conversely, the execution time for the built-in `cv2.filter2D` remained highly stable and was largely unaffected by the increased kernel dimensions.

```
=====
Performance Benchmark: Execution Time (ms)
=====
```

Algorithm	3x3 Kernel	15x15 Kernel
Custom (my_imfilter)	3617.66 ms	15816.31 ms
OpenCV (cv2.filter2D)	0.65 ms	77.56 ms

```
-----
```

Custom slowdown factor:	4.4x slower
OpenCV slowdown factor:	118.5x slower

```
=====
```

Conclusions and Discussion

Interpretation & Comparison:

The results strongly support the hypothesis: implementation choices dramatically impact execution time, even when outputs are identical. Python loops incur massive computational overhead compared to parallelized operations (Vectorization, LUT). Furthermore, the difference map confirms that industrial libraries like OpenCV achieve higher accuracy at image edges by employing advanced boundary conditions (reflection).

Limitations:

The $\mathcal{O}(K^2)$ complexity of the custom spatial implementation made it impractically slow during the stress test (15x15 kernel). Additionally, the Zero-Padding approach introduced artificial black pixels, causing mathematical distortions at the image boundaries compared to built-in library functions.

Conclusions & Future Work:

Implementing algorithms from scratch is crucial for understanding core mechanics (such as value clipping and data-type casting) and offers a significant advantage when highly specific algorithmic flexibility or custom logic is required beyond standard library capabilities. However, large-scale, real-world computer vision applications strictly necessitate the use of optimized libraries. Future work could focus on optimizing custom convolutions via Separable Filters or GPU acceleration.